System Design Interview
Introduction
Fundamental
Join the Community
Scale From Zero To Millions of Users
Back of the envelope Estimation
A Framework For System Design Interviews
Design a Rate Limiter
Design Consistent Hashing
Design a Key-value Store
Design a Unique ID Generator in Distributed Systems
Design a Link Shortener
Design a Web Crawler
Design a Notification System
Design A News Feed System
Design a Chat System
Design A Search Autocomplete System
Indexing Microservices and Monitoring Metrics
Ad Click Event Aggregation
Hotel Reservation System
Distributed Email Service
Ad-hoc Object Storage
Real-time Gaming Leaderboard
Payment Systems
Digital Wallet
Stock Exchange
The Learning Continues

21
Metrics Monitoring and Alerting System

In this chapter, we explore the design of a scalable metrics monitoring and alerting system. A well-designed monitoring and alerting system plays a key role in ensuring the high availability and reliability of internet-scale systems. We will cover the following topics:

Let's first look, understand the problem and establish the scope of the design before diving into the details.

Candidate: Who are the customers of this system? Is it for internal use only, for example, to monitor the servers of our company? Or, are we designing a SaaS service like Datadog, Dynatrace, SignalFx, etc.?
Interviewer: It is for internal use only. We manage all the servers by ourselves.
Candidate: Do we need to collect various metrics and logs?
Interviewer: Yes, we need to collect various operational metrics. These can be low-level usage data of the operating system, such as CPU usage, disk space, memory, etc. It can also be high-level business metrics. For instance, we want to know how many thousands per second of servers in the system are running. We also want business metrics, such as how many daily active users we have.
Candidate: What is the scale of the infrastructure we are monitoring with this system?
Interviewer: 100 million daily active users, 1000 server pools, and 100 machines per server.
Candidate: How long should we keep the data?
Interviewer: Let's assume we want to keep the data for one year.
Candidate: Can we reduce the resolution of the metrics data for long-term storage?
Interviewer: That is a good idea. We can keep a new metric for one minute for seven days, aggregated data for 7 days, after 7 days, you may roll them up to 1 minute resolution for 30 days, after 30 days, you may roll them up at an 1 hour resolution.
Candidate: What alert the supported alert channels?
Interviewer: Email, phone, PagerDuty, or webhooks (HTTP endpoints).
Candidate: Do we need to collect logs, such as error log or access log?
Interviewer: No.
Candidate: Do we need to support distributed system tracing?
Interviewer: No.

Step 1 - Understand the Problem and Establish Design Scope

A metrics monitoring and alerting system can mean many different things to different companies, so it is essential to ask clarifying questions to understand the requirements. For example, when you are asked to design a system that collects metrics, you need to ask what kind of metrics the company wants to collect. Is it server-level infrastructure metrics or not?

Let's first look, understand the problem and establish the scope of the design before diving into the details.

Figure 1 Metrics monitoring and alerting services

Fundamentals

A scalable monitoring and alerting system generally contains five components, as illustrated in Figure 2.

Figure 2 Components of the system

- Data collection: collect metrics data from different sources.
- Data transmission: transfer data from sources to the metrics monitoring system.
- Data storage: organize and store incoming data.
- Alerting: analyze incoming data, detect anomalies, and generate alerts. The system must be able to send alerts in different channels.
- Visualization: present data in graphs, charts, etc. Engineers are better at identifying patterns, trends, or problems when data is presented visually, so we need visualization functionality.

Data model

Metrics data is usually recorded as time series that contains a set of values with their associated timestamps. The series itself can be uniquely identified by its name, and optionally by a set of labels.

Let's take a look at the example.

Example 1: What is the CPU load of production server instance 9207 at 20:07?

Figure 3 Data model

The data point highlighted in Figure 3 can be represented by Table 1.

metric_name	cpu.load
labels	host=i9207,env=prod
timestamp	1613707265
value	0.29

Table 1 The data point represented in a specific format

In this example, the time series is represented by the metric name, the labels (host=i9207,env=prod), and a single point value at a specific timestamp.

Example 2: What is the average CPU load across all web servers in the us-east region for the last 10 minutes? Conceptually, we would put up something like this from storage where the metric name is "CPU.load" and the region label is "us-east":

CPU.load host=webserver01,region=us-east 1613707265:50
CPU.load host=webserver02,region=us-east 1613707265:62
CPU.load host=webserver03,region=us-east 1613707265:43

CPU.load host=webserver01,region=us-east 1613707265:51
CPU.load host=webserver02,region=us-east 1613707265:64
CPU.load host=webserver03,region=us-east 1613707265:59

The average CPU load could be computed by averaging the values in the last three lines. The format of the time-to-the-above example is called the line protocol. It is a common input format for many monitoring software in the market. Prometheus OK and OpenTSDB OK are two examples.

Every time series consists of the following [8]:

Name	Type
A metric name	String
A set of hashtables	List of key-value pairs
An array of values and their timestamps	An array of values, timestamp pairs

Table 2 Time series

Data access pattern

Figure 4 Data access pattern

In Figure 5, each label on the y-axis represents a time series uniquely identified by the metric and labels; while the x-axis represents time.

In the write load is heavy, this as can use, there can be many time series data points written at any moment. As we mentioned in the "High-level requirements" section, our system is to collect operational metrics to see what is going on. Most metrics are written in real-time, and the traffic is predictable in many ways.

At the same time, the read load is lastly. Both visualization and alerting services send queries to the database, and depending on the access patterns of the graphs and alerts, the read volume could be heavy.

In other words, the system is a write-heavy and read-heavy system.

Data storage system

The data storage system is the heart of the design. It's not recommended to build your own storage system or use a general-purpose database for storing metrics data. Although, OK for small-scale.

A general purpose database, in theory, could support time series data, but it's a general-purpose relatively. A relational database is not optimized for operations you would commonly perform against time series data. For instance, to calculate the exponential moving average over a time series, a relational database would need to write a complex query. Similar to the Map and Reduce, Similarly, to support aggregations, we need to add an index for each key. Moreover, a general-purpose relational database does not perform well under constant heavy write load. At our scale, we would need to expand significant time and resources to optimize a relational database.

Other least limited in many ways, a time series database is optimized for time series data. It performs very effectively. For example, Druid and Illyrioda DB. can be used for time series data. However, this would require deep knowledge of the internal workings of each DB. Instead, we can use a cloud-based time series database, such as Amazon Timestream. We focus on a time series database. Specifically, scalability, and performance Kafka DB, database is not scalable.

There are many storage systems available that are optimized for time series data. The optimization lets us use fewer servers to handle the same volume of data. Many of these databases also have custom functions specific, depending on the nature of the stored data that are much easier to use than SQL. Some most popular functions to manage data retention and data aggregation. Here are a few time series databases:

OpenTSDB is a distributed time series database. It's based on Hadoop and HBase, running in Hadoop/HBase. Cluster with Cassandra, Twitter uses BinaryDB [10], and Amazon offers Timestream as a time series database [11]. According to DB-engines [12], the most popular time series database is InfluxDB, followed by Prometheus. Both are designed to store large volumes of time series data and support fast, real-time queries. Both can run on Docker containers. Here are some of their pros and cons: Kafka DB, they both handle durability and performance quite well. As shown in Figure 5, at InfluxDB with 8 cores and 32GB RAM can handle over 250,000 writes per second.

	InfluxDB	Prometheus
2 Vcores	2 vCOR	1600
4 Vcores	4vcor	3000
8 cores	32c 16G	5000+

Figure 5 InfluxDB benchmarking

Since a time series database is a specialized database, you can not expected to understand the internals in an interview unless you explicitly mentioned it in your resume. For the purpose of an interview, it's important to understand the metrics data are time series in nature and that a time series database is a good fit for storing metrics data.

Another factor is if storing time series database in different engineers and analysis of a large amount of time series data is lastly, also known as large in volume datasets. For example, InfluxDB builds indexes in order to facilitate fast lookup of time series by labels. For instance, finding all time series in us-east-1 region, Prometheus uses a custom time series database. It's not a time series database. Instead, it's an SQL database. Kafka DB, it's not scalable. Having a wide set of possible network. This feature is critical for visualization, and it would take a lot of effort to build the with a general purpose database.

High-level design

The high-level design diagram is shown in Figure 6.

Figure 6 High-level design

- **Metrics source**: This can be application servers, SQL databases, message queues, etc.
- **Metrics collector**: It gathers metrics data and writes to time series database.
- **Time series database**: A time series database is a database specifically optimized for time series data. For instance, Druid, Illyrioda DB, and Amazon DB, Timestream are time series databases. Kafka DB, According to DB-engines [12], the most popular time series database is InfluxDB, followed by Prometheus. Both are designed to store large volumes of time series data and support fast, real-time queries. Both can run on Docker containers. Here are some of their pros and cons: they both handle durability and performance quite well. As shown in Figure 5, at InfluxDB with 8 cores and 32GB RAM can handle over 250,000 writes per second.
- **Query service**: The query service makes it easy to query and retrieve data from the time series database. This should be a very fast service, it adds a large amount of time series data. It makes it simple and fast to facilitate the fast lookup of time series data by labels.
- **Alerting system**: This sends alert messages to various alert channels. Alert from the time series database. This should be a very fast service, it adds a large amount of time series data. It makes it simple and fast to facilitate the time series database's fast query efficiency.
- **Alerting system**: This sends alert notifications to various alerting destinations.
- **Visualization system**: This shows metrics in the form of various graph/charts.

Step 3 - Design Deep Dive

In a system design interview, it is unrealistic or wasteful to dive deep into a few key components in there. In this section, we investigate the following topics in detail:

- Metrics collection
- Scaling the metrics monitoring system
- Query service
- Storage layer
- Alerting system
- Visualization system

Metrics collection

For metrics collection, for example, CPU usage, occasional data loss is not the end of the world. It's acceptable for clients to fail and forget. Now let's take a look at the metrics collection flow. This part of the system is made the standard flow (Figure 7).

Figure 7 Metrics collection flow

Pull vs push models

There are two ways metrics data can be collected, pull or push. It is a routine debate as to which one is better and there is no clear answer. Let's take a look first.

Pull model

Figure 8 shows data collection with a pull model. Here HTTP We have dedicated metrics collectors which pull metrics values from the running applications continuously.

Figure 8 Pull model

In this approach, the metrics collector needs to know the complete list of service endpoints to pull data from. One naive approach is to set up a file to hold DNSHP information for every service endpoint on the metrics collector' servers. When the data is stale, this approach is hard to maintain. In a large-scale distributed environment, servers are added or removed frequently, and we want to ensure that metrics are not missed because of inaccurate information. Service Discovery provides a centralized service for service discovery. It maintains a solution available through Service Discovery, provided by etcd, Zookeeper, SD, etc., clients of services register their availability and the metrics collector can be notified by the Service Discovery component whenever the list of service endpoints changes.

Service discovery contains configuration rules about when and where to collect metrics as shown in Figure 9.

Figure 9 Service discovery

Figure 10 explains the pull model in detail.

Figure 10 Pull model

2. The metrics collector fetches configuration metadata of service endpoints from Service Discovery. Metadata includes (port/http method, IP addresses, timeout and retry parameters, etc.
3. The metrics collector pulls metrics data from objects attained HTTP endpoint (for example, /metrics). To expose the endpoint, a client library usually needs to add the module to the service. In Figure 11, Prometheus client libraries.
4. Optionally, the metrics collector registers a change event notification with Service Discovery to receive an update whenever the service endpoints. Alternatively, the metrics collector can pull the collection manage periodically.

At our scale, a single metric collector will not be able to handle thousands of servers. We must use a pool of metrics collectors to handle the workload. A common problem when using a pool of distributed metrics collectors is to ensure that it is the most efficient use of resources and prevent jobs conflict. There must exist some coordination of servers among the instances in such cases.

One potential approach is to designate each collector to a single or a consistent hash ring, and then map every single server being monitored to a collector using the hash ring. This ensure every server is assigned to one collector and that every collector knows what to scrape.

As shown in Figure 12, there are four collectors and six metrics source servers. Each collector is responsible for collecting metrics from a distinct set of servers. Collector 3 is responsible for collecting metrics from "server pool" 1 and server 3.

Figure 12 Consistent hashing

Push model

As shown in Figure 13, A push model various metrics sources, such as web servers, database servers, etc., directly send metrics to the metrics collector.

Figure 13 Push model

In a push model, a collector agent is commonly installed on every server being monitored. A collector agent is a piece of long-running software that accepts incoming metrics over a protocol that supports a push model. For instance, Amazon Cloudwatch agent, the collector agent may also aggregate metrics (especially in simple aggregation forms), before sending them to metrics collectors.

Aggregation is an effective way to reduce the volume of data sent to the metrics collector. If the push traffic is high and the metrics collector is overwhelmed, it may be unable to accept data from the collector agent. In this case, the data is lastly. If the collector agent times out, it might retry multiple times. If the server pool is at a very scale and many clients send data at the same time, it may overwhelm the backend. Some lastly, short-lived lastly might send to data less when the metrics collector falls behind.

To prevent the metrics collector's being overwhelmed, a large enough limit for metrics collector should be in place scaling cluster with a fast enough to handle the incoming traffic. The cluster should scale up and down based on the CPU load of the metrics collector servers.

Figure 14 Pull vs push model

Pull vs push?

So, What's best is the better choice for us? Just like many things in life, there is no clear answer. Both sides have widely adopted each world. Pull model:

- Examples of pull architecture include Prometheus.
- Examples of push architecture include Amazon Cloudwatch [13] and Graphite [14].

Among the advantages and disadvantages of each approach is clear, it doesn't matter that pulling a server during an interview. Table 3 compares the pros and cons of pull and pull architectures [16] [18] [20] [21].

	Pull	Push
Easy debugging	The metrics endpoint on application servers used for pulling data can be used to view metrics. If we want to see the metrics on a specific server, we can do this on your laptop.	Pull model can be used for metrics boxes.
Health check	If an application server doesn't respond to the pull, it quickly knows that it's an application server. Issue.	If the metrics collector doesn't receive metrics, the problem might be caused by network issues.
Short-lived jobs	Some of the short-lived jobs might be lost before they are scraped enough to be pulled. Metrics. This can be fixed by introducing push gateway for the pull model.	Short-lived jobs can push metrics to the metrics collector.
Firewall or complicated network setups	Having servers pulling metrics requires all metrics endpoints to multiple data centers. This is a potential issue. It requires a more elaborate network configuration.	If the metrics collector is set up with a load balancer and an authentication proxy, it is possible to receive data from anywhere. Push metrics.
Performance	Pull methods typically use TCP.	Push methods typically use UDP. This means the push method provides lower latency. However, the counterargument here is that the effort of establishing a TCP connection is small compared to collecting the metrics data.
Data authenticity	Application servers to collect metrics from are defined in advance in configuration. Metrics received from those servers are guaranteed to be authentic.	Any kind of client can send metrics to the metrics collector. The push model can be hardening servers from rogue metrics or spoofing metrics.

Table 3 Pull vs push

As mentioned above, pull or push is a routine debate topic and there is no clear answer. A large organization probably needs to support both systems. Let's look at some ways to handle some of the issues. There might not be a way to install an agent from which to push data or the client might be firewalled from the metrics collector.

Scale the metrics transmission pipeline

Figure 15 Metrics transmission pipeline

Let's zoom in on the metrics collector and time series database. Whether you use the push or pull model, the metrics collector is a cluster of servers, and the cluster must be able to handle a large volume of metrics data. The time series database backs the metrics collector cluster in our system. We need to ensure that there are an adequate number of servers in the cluster to handle the demand.

However, there is a risk of data loss if the time series database is overwhelm. To mitigate this problem, we introduce a queuing component as shown in Figure 16.

Figure 16 Kafka system

In this design, the metrics collector sends metrics data to queuing systems like Kafka. They consumers or streaming processing services such as Storm, Flink, and Spark, process and push data to the time series database. This approach has several advantages:

- Kafka is used as a highly reliable and scalable distributed messaging platform.
- It decouples data collection and processing services from each other.
- It can easily prevent data loss from the metrics collector by retaining the data in Kafka.

Scale through Kafka

There are a couple of ways that we can leverage Kafka's built-in partition mechanism to scale our system.

- Configure the number of partitions based on throughput requirements.
- Partition metrics data by metric name, or hostname, or tags (tags contain information that is

Figure 17 Kafka partition

- Further partition metrics data by subfields.
- Categorize and prioritize metrics so that important metrics can be processed first.

Alternative to Kafka

Maintaining a production-scale Kafka cluster system is for small undertaking. You might get pushback from the interviewer that metrics data are large-scale monitoring ingestion systems in use without using an intermediate queue. Facebook's Gorilla [24] in-memory time series database is a prime example. It is designed to remain highly available for writes, even when there is a partial network failure. It could be argued that such a design is as reliable as having an intermediate queue like Kafka.

Where aggregations can happen

Metrics can be aggregated in different places: in the collection agent (on the client-side), the ingestion pipeline (before writing to storage), and the query side (after writing to storage). Let's take a closer look at each of these.

Collection agent - The collection agent installed on the client-side only supports simple aggregation logic. For example, aggregate a counter every minute before it is sent to the metrics collector.

Ingestion pipeline. To aggregate data before writing to the storage, we usually need stream processing engines such as Flink. The write volume will be significantly reduced since only the calculated result is written to the database. However, handling late-arriving events could be a challenge and another downside is that we lose data precision and some flexibility because we no longer store the raw data.

Query side. Raw data can be aggregated over a given time period at query time. There is no data loss with this approach, but the query speed might be slower because the query result is computed at query time and is run against the whole dataset.

Query service

The query service comprises a cluster of query servers, which access the time-series databases and handle requests from the visualization or alerting systems. Having a dedicated set of query servers decouples time-series databases from the clients (visualization and alerting systems). And this gives us the flexibility to change the time-series database or the visualization and alerting systems, whenever needed.

Cache layer

To reduce the load of the time-series database and make query service more performant, cache servers are added to store query results, as shown in Figure 17.

Figure 17 Cache layer

The case against query service

There might not be a pressing need to introduce our own abstraction (a query service) because most industrial-scale visual and alerting systems have powerful plugins to interface with well-known time-series databases on the market. And with a well-chosen time-series database, there is no need to add our own query layer.

Time-series database query language

Most popular metrics monitoring systems like Prometheus and InfluxDB don't use SQL and have their own query languages. One major reason for this is that it is hard to express SQL queries for time series data. For example, as mentioned here [25], computing an exponential moving average might look like this in SQL:

```
select id,
       temp,
       avg(temp) over (partition by group_nr order by time_read) as rolling_avg
from (
    select id,
           temp_read,
           time_read,
           interval_group,
           id - row_number() over (partition by interval_group order by time_read) as group_nr
    from (
        select id,
               temp_read,
               time_read,
               'epoch'::timestamp + '900 seconds'::interval * (extract(epoch from time_read)::int / 900) as interval_group,
               battery,
               from readings
        ) t1
    ) t2
order by time_read;
```

While in Flux, a language that's optimized for time-series analysis (used in InfluxDB), it looks like this. As you can see, it's much easier to understand.

```
from(db:"telegraf")
  |> range(start:-1h)
  |> filter(fn: (r) => r._measurement == "foo")
  |> exponentialMovingAverage(size:-10s)
```

Storage layer

Now let's dive into the storage layer.

Choose a time-series database carefully

According to a research paper published by Facebook [24], at least 85% of all queries to the operational data store were for data collected in the past 26 hours. If we use a time-series database that harnesses this property, it could have a significant impact on overall system performance. If you are interested in the design of the storage engine, please refer to the design document of the Influx DB storage engine [26].

Space optimization

As explained in high level requirements, the amount of metric data to store is enormous. Here are a few strategies for tackling this.

Data encoding and compression

Data encoding and compression can significantly reduce the size of data. These features are usually built into a good time-series database. Here is a simple example.

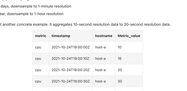

Figure 18 Data encoding

As you can see in the image above, 1610087371 or 1610087381 differ by only 10 seconds, which takes only 4 bits to represent, instead of the full timestamp of 32 bits. So, rather than storing absolute values, the delta of the values can be stored along with one base value like 1610087371, 10, 10, 9, 11!

Downsampling

Downsampling is the process of converting high-resolution data to low-resolution to reduce overall disk usage. Since our data retention is 1 year, we can downsample old data. For example, we can let engineers and data scientists define rules for different metrics. Here is an example:

- Retention: 7 days, no sampling
- Retention: 30 days, downsample to 1-minute resolution
- Retention: 1 year, downsample to 1-hour resolution

Let's take a look at another concrete example. It aggregates 10-second resolution data to 30-second resolution data.

metric	timestamp	hostname	Metric_value
cpu	2021-10-24T19:00:00Z	host-a	10
cpu	2021-10-24T19:00:10Z	host-a	16
cpu	2021-10-24T19:00:20Z	host-a	20
cpu	2021-10-24T19:00:30Z	host-a	30
cpu	2021-10-24T19:00:40Z	host-a	20
cpu	2021-10-24T19:00:50Z	host-a	30

Table 4 10-second resolution data

Rollup from 10-second resolution data to 30-second resolution data.

metric	timestamp	hostname	Metric_value (avg)
cpu	2021-10-24T19:00:00Z	host-a	19
cpu	2021-10-24T19:00:30Z	host-a	25

Table 5 30-second resolution data

Cold storage

Cold storage is the storage of inactive data that rarely used. The financial cost for cold storage is much lower.

In a nutshell, we should probably use third-party visualization and alerting systems, instead of building our own.

Alerting system

For the purpose of the interview, let's look at the alerting system, shown in Figure 19 below.

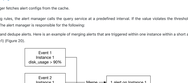

Figure 19 Alerting system

The alert flow works as follows:

1. Load config files to cache servers. Rules are defined as config files on the disk. YAML [27] is a commonly used format to define rules. Here's an example of alert rules:

```
- name: instance_down
  rules:

  # Alert for any instance that is unreachable for >5 minutes.
  - alert: instance_down
    expr: up == 0
    for: 5m
    labels:
      severity: page
```

2. The alert manager fetches alert configs from the cache.

3. Based on config rules, the alert manager calls the query service at a predefined interval. If the value violates the threshold, an alert event is created. The alert manager is responsible for the following:

- Filter, merge, and dedupe alerts. Here is an example of merging alerts that are triggered within one instance within a short amount of time (instance1 (Figure 20).

Figure 20 Merge alerts

- Access control. To avoid human error and keep the system secure, it is essential to restrict access to certain alert management operations to authorized individuals only.
- Retry. The alert manager checks alert states and ensures a notification is sent at least once.

4. The alert store is a key-value database, such as Cassandra, that keeps the state (inactive, pending, firing, resolved) of all alerts. It ensures a notification is sent at least once.

5. Eligible alerts are inserted into Kafka.

6. Alert consumers pull alert events from Kafka.

7. Alert consumers process alert events from Kafka and send notifications over to different channels such as email, text message, PagerDuty, or HTTP endpoints.

Alerting system – build vs buy

There are many industrial-scale alerting systems available off-the-shelf, and most provide tight integration with the popular time-series databases. Many of these alerting systems integrate well with existing notification channels, such as email and PagerDuty. In the real world, it is a tough call to justify building your own alerting system. In interview settings, especially for a senior position, he ready to justify your decision.

Visualization system

Visualization is built on top of the data layer. Metrics can be shown on the metrics dashboard over various time scales and alerts can be shown on the alerts dashboard. Figure 21 shows a dashboard that displays some of the metrics like the current server requests, memory/CPU utilization, page load time, traffic, and login information [28].

Grafana Play Home
server requests
Memory / CPU
page load time
traffic
login
mean server load per region
mem_free
mem_cached
mem_buffered
cpu_user
cpu_system
cpu_idle
cpu_nice
cpu_aidle
cpu_wio
cpu_steal
bytes_in
bytes_out
pkts_in
pkts_out
cpu_intrpt
cpu_sintrpt
cpu_interrupt
cpu_softirq
cpu_aintrpt
pkts_broadcast
pkts_multicast
boottime
sys_clock
machine_type
swap_free
swap_total
mtu
gexec
machine_type
os_name
os_release
mem_shared
cpu_num
cpu_speed
heartbeat
location
tcpxmitq
tcpofoq
tcpcongestion
tcpretrans
tcpsyncookies
tcpextra
tcpaddrbacklog
tcpbacklog
tcpreordering
tcpdelivery
tcpfastopen
tcpfastopenlisten
tcpfastopenactive
tcpfastopenactivefail
tcpfastopenpassive
tcpfastopenpassivefail
tcpfastopencookiereqd
tcpfastopenblackhole
tcpdsack
tcpsackrecovery
tcpsackreneging
tcpsackreorder
tcpsackshift
tcpsackshiftfallback
tcprcvq
tcpprunecalled
tcpprunedosomuch
tcptimeouts
tcpdelayedack
tcpdelayedacklocks
tcpemptyrcvqueue
tcpktimeout
tcpmemorypressures
tcpmemorypressuretimes
tcpfromzerowindow
tcptozerowindow
tcpzeroretransmit
tcpmtudisc
tcpmtudiscfail
tcpmtuprobe
tcpmtuprobefail
tcpprequeued
tcpprequeuedropped
tcpprequeueprocessed
tcpdirectcopyfromprequeue
tcpprequeueprocessedbyhandler
tcpprequeueprocessedbyhandlerbacklog
tcpprequeueprocessedbyhandlerdrop
tcpprequeueprocessedbyhandlerquickack
tcpprequeueprocessedbyhandlerwaitack
tcpprequeueprocessedbyhandlerwaitackfast
tcpprequeueprocessedbyhandlerwaitackslow
tcpprequeueprocessedbyhandlerwaitackslowtotal
tcpprequeueprocessedbyhandlerwaitackslowavg
tcpprequeueprocessedbyhandlerwaitackslowmax
tcpprequeueprocessedbyhandlerwaitackslowmin
tcpprequeueprocessedbyhandlerwaitackslowcount
tcpprequeueprocessedbyhandlerwaitackslowpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentage
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotal
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavg
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmax
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgmin
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgcount
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercentagetotalavgpercent
tcpprequeueprocessedbyhandlerwaitackslowpercentagetotalavgpercentagetotalav